REVISIÓN DE SEGURIDAD · TESINA · PARTE 3

El mismo número, dos veces

Una revisión manual de las tres apps del sistema (más allá del `npm audit`) encontró una condición de carrera real en cómo `votacion-backend` firma transacciones en BFA. El primer arreglo pareció andar, pero una prueba con más concurrencia lo desmintió — esta es la bitácora de cómo se encontró, se probó y se corrigió de verdad.

Fecha: **2026-09-07** Alcance: **votacion-backend** · blockchain.service.ts

Método: **revisión de código + votos concurrentes reales contra BFA**

1	2	3	3/3
Hallazgo crítico nuevo	Intentos de arreglo	Pruebas de concurrencia real	Votos simultáneos ok, prueba final

01 · CONTEXTO

De dónde salió esto

Además de las tres cadenas de `npm audit` ya resueltas (ver informes anteriores), se pidió una revisión manual de flujos y lógica en las tres apps. Aparecieron varios puntos — la mayoría ya identificados en sesiones previas y con decisión tomada de *documentar, no mitigar* (quedan fuera del alcance de este informe, que se enfoca en el hallazgo nuevo):

HALLAZGO	ESTADO	NOTA
Voto fantasma por firmante único (el backend puede calcular el hash de cualquier votante sin su clave)	DOCUMENTADO	Limitación estructural, ya conversada; queda para la sección de trabajo futuro de la tesina.
BD/Redis sin rol de mínimo privilegio (rol `owner` de Neon)	DOCUMENTADO	Mitigación propuesta, no implementada.
DoS económico: un admin puede vaciar el	DOCUMENTADO	Requiere un límite de negocio, no técnico.

saldo de gas creando elecciones sin límite		
Confianza en que el nodo RPC de BFA no mienta	DOCUMENTADO	Inherente a blockchain permisionada (consorcio), no a un bug.
Inyección de fórmulas Excel en los CSV exportados (padrón/participación)	MENOR	Riesgo bajo: los datos vienen del padrón que carga el propio admin, no de un input público.
Pool de conexiones de Prisma sin límite explícito (usa el default por CPU de la máquina)	MENOR	Se agotó en vivo durante las pruebas de concurrencia de este informe ("connection limit: 13"). Antes de escalar a los ~8000 votantes reales, fijarlo a mano (`connection_limit`/`pool_timeout` en `DATABASE_URL`) en vez de dejarlo en el valor que le toque a la máquina.
Condición de carrera en las transacciones a BFA	ARREGLADO Y PROBADO	Este informe.

02 · EL HALLAZGO

Una sola wallet, todas las transacciones

`blockchain.service.ts` tiene una única instancia de `ethers.Wallet` que firma *todo* — cada voto y cada deploy de elección nueva. Cada transacción necesita un número de secuencia (**nonce**): la primera que manda esa wallet es la 0, la siguiente la 1, y así — la red lo usa para saber el orden y no procesar la misma transacción dos veces.

> 📖 Es como los números de cheque de una chequera: cada cheque nuevo lleva el siguiente número, en orden, sin repetir. Si dos personas escriben un cheque "número 47" de la misma chequera casi al mismo tiempo, el banco solo puede aceptar uno — el otro rebota.

El código no tenía ningún control de concurrencia (confirmado por grep: cero resultados de "nonce"/"mutex"/"queue" en todo el backend antes de este cambio) — y el caso de uso normal del sistema es exactamente el que dispara el problema: muchos votantes votando en la misma ventana horaria.

03 · BITÁCORA

Dos intentos, tres pruebas reales

Cada prueba se hizo disparando votos de verdad, en simultáneo, contra la blockchain real de BFA — no hay forma de ver esto con curls uno atrás del otro.

1

Intento 1: una cola que solo ordena los pedidos

Se agregó `encolarEnvio()`: fuerza a que el paso de firmar+enviar nunca se solape entre llamadas, para que nunca haya dos pidiéndole el nonce a la red al mismo tiempo. La confirmación (`tx.wait()`) sigue en paralelo — costo en velocidad mínimo.

2

Prueba A — 2 votos concurrentes (Ana + Lucía) INCONCLUSA

Ana votó bien. Lucía recibió "ya emitiste tu voto" en 883ms — sospechosamente rápido para una confirmación real (la de Ana tardó 234 segundos). Al consultar el contrato directamente, **ambos hashes figuraban como "ya votó"** — pero resultó ser un dato viejo: Lucía tenía un comprobante borrado a mano en una sesión de pruebas anterior. No probaba ni desmentía el arreglo.

3

Prueba B — 3 votos concurrentes, elección de prueba nueva FALLÓ

Con tres votantes frescos (nunca habían votado en esa elección), disparados exactamente al mismo tiempo. 2 de 3 se registraron bien — el tercero falló con un error real de la red:

> **code=REPLACEMENT_UNDERPRICED**
> message: "replacement transaction underpriced"

Esto confirma que el intento 1 no alcanzaba: BFA es una red de consorcio, probablemente con más de un nodo detrás de un balanceador — el nodo que atendió al segundo pedido no se había enterado todavía del primero, y le devolvió el mismo número aunque los pedidos ya llegaran ordenados de nuestro lado.

4

Intento 2: contar el nonce nosotros mismos, en memoria

En vez de preguntarle a la red "¿cuál es el próximo número?" en cada envío, se sincroniza una sola vez al arrancar (o de nuevo si algo falló) y después se cuenta localmente, sin volver a confiar en que el nodo esté al día:

src/services/blockchain.service.ts

```
let siguienteNonce: number | null = null;
```

```
function encolarEnvio(construirTx) {
  return colaEnvio.then(async () => {

    // antes: preguntar "pending" a la red en cada envio

    if (siguienteNonce === null) {
      siguienteNonce = await provider.getTransactionCount(BACKEND_ADDRESS, 'pend
    }
    const nonce = siguienteNonce;
    const resultado = await construirTx(nonce);   // { nonce } explicito
    siguienteNonce = nonce + 1;

    return resultado;
  });
}
```

5 Prueba C — 3 votos concurrentes, otra elección nueva ÉXITO

Misma prueba que la B, con el arreglo nuevo. Los tres votos, disparados en el mismo instante:

```
ANA> {"mensaje":"Voto registrado correctamente", ...} 201
LUCIA> {"mensaje":"Voto registrado correctamente", ...} 201
JUAN> {"mensaje":"Voto registrado correctamente", ...} 201
```

Tres tx_hash distintos, sin ningún `REPLACEMENT_UNDERPRICED`. Se confirmó además que quedó reflejado del lado de la base (`ya_voto: true` para los tres).

04 · UNA PREGUNTA QUE VALE LA PENA RESPONDER

¿Y si alguien más usa BFA al mismo tiempo?

¿Otra persona usando la red en el medio de nuestros votos nos puede volver a romper el conteo?

No. El nonce es **por cuenta**, no global a la red — cada wallet lleva su propia numeración, como cada chequera tiene la suya. Lo único que sí rompería nuestro contador es que **esa misma wallet del backend** mandara una transacción por afuera de este archivo (un script aparte, una segunda instancia del backend) — mientras `blockchain.service.ts` siga siendo el único punto que la usa, no hay riesgo.

05 · CIERRE

Estado final

ARCHIVO MODIFICADO	CAMBIOS EN LA API / CONTRATOS
1 — `blockchain.service.ts`	Ninguno
VERIFICADO CON	**LIMITACIÓN CONOCIDA**
3 pruebas de concurrencia real contra BFA	Serializa dentro de un proceso; con más de una instancia del backend, se repetiría (necesitaría un lock compartido, ej. Redis)